Programación web con .NET (IFCD59) – Módulo 1: conceptos básicos de programación e introducción a .NET.

APUNTES. LOS METODOS

1. ¿Qué es un método?

Un método es un bloque de código con un nombre, que realiza una tarea concreta.

Sirve para organizar el programa en "piezas" reutilizables, igual que en una receta separas pasos: preparar ingredientes, cocinar, servir, etc.

En C#, los métodos suelen vivir dentro de una clase. En una app de consola básica, ya has usado el método Main, que es el punto de entrada del programa.

2. ¿Por qué usamos métodos?

- Reutilización: escribes una vez y lo usas muchas veces.
- Orden y lectura: el código se entiende mejor si está separado por tareas.
- Mantenimiento: si una regla cambia, modificas un solo método.
- Pruebas: es más fácil probar (testear) métodos pequeños que un bloque enorme.

3. Partes de un método (firma)

Un método típico se define con:

```
modificador tipoRetorno NombreMetodo(parámetros) { ... }
```

Ejemplo:

```
public static int Sumar(int a, int b) { return a + b; }
```

4. Parámetros y retorno

• Parámetros: datos que le pasas al método para que trabaje con ellos.

• Retorno(**return**) valor que el método devuelve al final (si no devuelve nada, usamos la palabra **void**).

Método que devuelve un valor (int):

```
static int Cuadrado(int n)
{
    return n * n;
}
```

Método que no devuelve valor (void):

```
static void Saludar(string nombre)
{
    Console.WriteLine($"Hola, {nombre}");
}
```

5. Llamar (invocar) un método

Para usar un método en tu código, lo "llamas" por su nombre, pasándole los parámetros que necesite:

```
int r = Cuadrado(5);            // r = 25


Saludar("Marta");              // imprime: Hola, Marta
```

6. Métodos estáticos vs métodos de instancia

- static: pertenece a la clase. Puedes llamarlo sin crear un objeto.
- instancia (sin static): pertenece a un objeto concreto. Necesitas crear una instancia.

Ejemplo rápido:

```
class Calculadora
{
    public static int Sumar(int a, int b) => a + b; // estático

    public int Multiplicar(int a, int b) => a * b;  // de instancia
}

// Uso:


int s = Calculadora.Sumar(2, 3);



var calc = new Calculadora();
int m = calc.Multiplicar(2, 3);
```

7. Sobrecarga (mismo nombre, distintos parámetros)

C# permite tener varios métodos con el mismo nombre, siempre que cambien los parámetros (cantidad o tipos). Eso se llama sobrecarga.

```
static int Sumar(int a, int b) => a + b;
static double Sumar(double a, double b) => a + b;
static int Sumar(int a, int b, int c) => a + b + c;
```

8. Parámetros opcionales y valores por defecto

Puedes dar un valor por defecto a un parámetro, y entonces se vuelve opcional:

```
static void MostrarMensaje(string texto, int repeticiones = 1)
{
    for (int i = 0; i < repeticiones; i++)
        Console.WriteLine(texto);
}



// Uso:
MostrarMensaje("Hola");        // Se imprime 1 vez
MostrarMensaje("Hola", 3);     // Se imprime 3 veces
```

9. Parámetros por referencia: ref y out

Normalmente, los parámetros se pasan “por valor” (el método recibe una copia). Con ref/out puedes permitir que el método modifique la variable original.

Ejemplo con out (muy típico para TryParse):

```
string texto = "123";
bool ok = int.TryParse(texto, out int numero);

if (ok)
    Console.WriteLine(numero); // 123
```

10. Buenas prácticas (para empezar)

- Un método = una responsabilidad (programalos pequeños si es posible. Es más fácil depurar).

- Nombres claros: verbo + qué hace (CalcularTotal, ObtenerUsuario, ValidarEmail).

- Evita repetir código: si copias/pegas, probablemente necesitas un método.
- Parámetros mínimos: pasa solo lo necesario.

11. Ejemplo completo (consola) usando métodos

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        var notas = new List<int> { 6, 8, 5, 9 };
        double media = CalcularMedia(notas);
        Console.WriteLine($"Media: {media}");
        MostrarAprobado(media);
    }

    static double CalcularMedia(List<int> notas)
    {
        int suma = 0;
        foreach (var n in notas) suma += n;
        return (double)suma / notas.Count;
    }

    static void MostrarAprobado(double media)
    {

            if (media >= 5)

            {

                  Console.WriteLine("Aprobado");

            }

            else

            {

                  Console.WriteLine("Suspenso");

            }

    }
```